

• SOLUÇÃO
TECNOLOGICA

Tech Do Bem

Hugo Souza de Jesus

RM568542

Lucas Campanha dos Santos

RM566815

Lucas Marcelino Pompeu

RM567010

Documentação — Tech do Bem — Sprint 4 Java

Disciplina: Domain Driven Design Using Java **Curso:** 1TDSPP — FIAP **Equipe:**

RM	Nome
RM568542	Hugo Souza de Jesus
RM566815	Lucas Campanhã dos Santos
RM567010	Lucas Marcelino Pompeu

Sumário

1. [Objetivo e escopo do projeto](#)
2. [Stack e arquitetura em camadas](#)
3. [Camada Modelo \(entities\)](#)
4. [Camada DAO — acesso a dados](#)
5. [Camada BO — regras de negócio](#)
6. [Camada Resource — API REST](#)
7. [Tabela completa de endpoints](#)
8. [Protótipo — telas do sistema](#)
9. [Tratamento de exceções](#)
10. [Conexão com o banco de dados](#)
11. [Procedimentos para rodar a aplicação](#)
12. [Atendimento à rubrica](#)

1. Objetivo e escopo do projeto

A **Tech do Bem** é uma ONG de atendimento odontológico gratuito a populações vulneráveis. Este módulo é o **backend Java** da plataforma de gestão — uma API REST construída com **Quarkus 3.15** sobre **JDK 21**, conectando-se ao Oracle FIAP via JDBC. Cobre:

- **Autenticação** unificada de colaboradores e dentistas.
- **CRUD completo** de todos os recursos do domínio: colaboradores, dentistas, pacientes, campanhas, atendimentos, exames, notificações, anotações, solicitações.
- **Regras de negócio especiais**: aprovação de solicitação externa cria automaticamente um colaborador; notificações suportam dois fluxos (col→den e den→pac); anotações são polimórficas (autor × sobre).
- **Endpoints administrativos**: reconstrução do banco a partir de um seed embarcado e snapshot de relatórios.

A API é consumida pelo frontend React+Vite e expõe os dados também para a página de mapa que integra com o Backend_AI (modelo preditivo de demanda).

1.1 Limites do sistema

- **Inclui**: persistência, regras de negócio, validação de domínio, exposição REST.
- **Não inclui**: UI (delegada ao Frontend), ML (delegado ao Backend_AI), autenticação baseada em token (mantida simples por requisito acadêmico — login retorna o usuário e o frontend mantém sessão local).

2. Stack e arquitetura em camadas

2.1 Stack tecnológica

Camada	Tecnologia
Linguagem	Java 21 (Eclipse Temurin)
Framework	Quarkus 3.15.1
API REST	JAX-RS (quarkus-rest-jackson)

Serialização	Jackson
Persistência	JDBC puro (sem ORM)
Driver	com.oracle.database.jdbc:ojdbc17:23.26.1.0.0
Banco	Oracle 19c (FIAP — oracle.fiap.com.br:1521/orcl)
Build	Maven 3.9
Container	Docker multi-stage (Eclipse Temurin 21)

2.2 Organização em pacotes

```
br.com.fiap
├── entities/      classes modelo (9 entidades do domínio)
├── dao/           acesso a dados via JDBC (13 DAOs)
├── bo/            regras de negócio (12 BOs)
├── dto/           records Request/Response
├── resource/      endpoints REST (12 resources)
├── conexoes/      ConexaoFactory (DriverManager)
└── exceptions/   DadoInvalidoException, PersistenciaException
```



O fluxo de uma requisição obedece à separação clássica: **Resource (HTTP) → BO (regra) → DAO (SQL) → Connection (JDBC) → Oracle**. A camada BO valida entrada, mapeia DTO ↔ entidade, e delega a persistência aos DAOs.

3. Camada Modelo (entities)

9 classes modelo correspondendo às 9 tabelas principais do schema Oracle, com atributos privados, getters/setters, construtores e padrões de encapsulamento. A rubrica exige no mínimo 6 — entregamos 9.

Classe	Tabela	Atributos principais
Colaborador	T_COLABORADOR	id, nome, cpf, email, senha, cargo, disponibilidade
Dentista	T_DENTISTA	id, nome, cpf, email, senha, cro, especialidade, disponibilidade,

		idColaborador
Paciente	T_PACIENTE	id, nome, cpf, dataNasc, telefone, email, idDentista, cep, logradouro, bairro, cidade, uf, latitude, longitude
Campanha	T_CAMPANHA	id, nome, local, dataInicio, dataFim, idColaborador
Atendimento	T_ATENDIMENTO	id, data, tipo, status, observacoes, idPaciente, idDentista, idCampanha
Exame	T_EXAME	id, tipo, requisitos, resultado, idAtendimento
Notificacao	T_NOTIFICACAO	id, mensagem, dataEnvio, statusEnvio, canal, dataLeitura, idColaborador, idDentista, idPaciente
Solicitacao	T_SOLICITACAO	id, tipo, descricao, status, dataSolicitacao, idSolicitante, idRevisor, dataRevisao, comentarioRevisao, dados externos
Anotacao	T_ANOTACAO	id, texto, data, autorId, autorTipo, sobreTipo, sobreId

3.1 Casos polimórficos (modelagem cuidadosa)

- **Notificacao** suporta dois fluxos via FKs anuláveis:
 - `col_to_den`: `idColaborador` preenchido, `idPaciente == null`.
 - `den_to_pac`: `idPaciente` preenchido, `idColaborador == null`.
- **Solicitacao** distingue acesso interno (`idSolicitante` preenchido) de cadastro externo (`idSolicitante == null`, campos `*Externo` preenchidos).
- **Anotacao** é polimórfica: `autorTipo` $\in \{ colaborador, dentista \}$ e `sobreTipo` $\in \{ dentista, paciente, atendimento \}$.

4. Camada DAO — acesso a dados

13 DAOs implementam o CRUD via JDBC puro (`PreparedStatement`, `ResultSet`). Todos seguem o mesmo padrão:



```
public class XxxDAO {  
    public Connection minhaConexao;  
    public XxxDAO() throws SQLException, ClassNotFoundException {  
        minhaConexao = new ConexaoFactory().conexao();  
    }  
    public void inserir(Xxx x) throws SQLException;  
    public List<Xxx> selecionar() throws SQLException;  
    public void atualizar(Xxx x) throws SQLException;  
    public void deletar(int id) throws SQLException;  
}
```

4.1 DAOs principais (entidades)

DAO	Operações suportadas
ColaboradorDAO	CRUD + busca por email/senha (autenticação)
DentistaDAO	CRUD + busca por email/senha
PacienteDAO	CRUD + listagem com filtro geo (lat/lng não nulos)
CampanhaDAO	CRUD
AtendimentoDAO	CRUD + joins paciente/dentista/campanha
ExameDAO	CRUD vinculado a atendimento
NotificacaoDAO	CRUD + marcar como lida (UPDATE data_leitura)
SolicitacaoDAO	CRUD + revisão (aprovação/rejeição)
AnotacaoDAO	Inserção + listagem por destinatário + remoção

4.2 DAOs associativas

DAO	Tabela associativa	Função
CampanhaAtendimentoDAO	T_CAMPANHA_ATENDIMENTO	Histórico campanha × atendimento
ExameAtendimentoDAO	T_EXAME_ATENDIMENTO	Vínculo exame × atendimento
		Histórico de envios

EnviaDAO

T_ENVIA

de notificações

4.3 Boas práticas adotadas

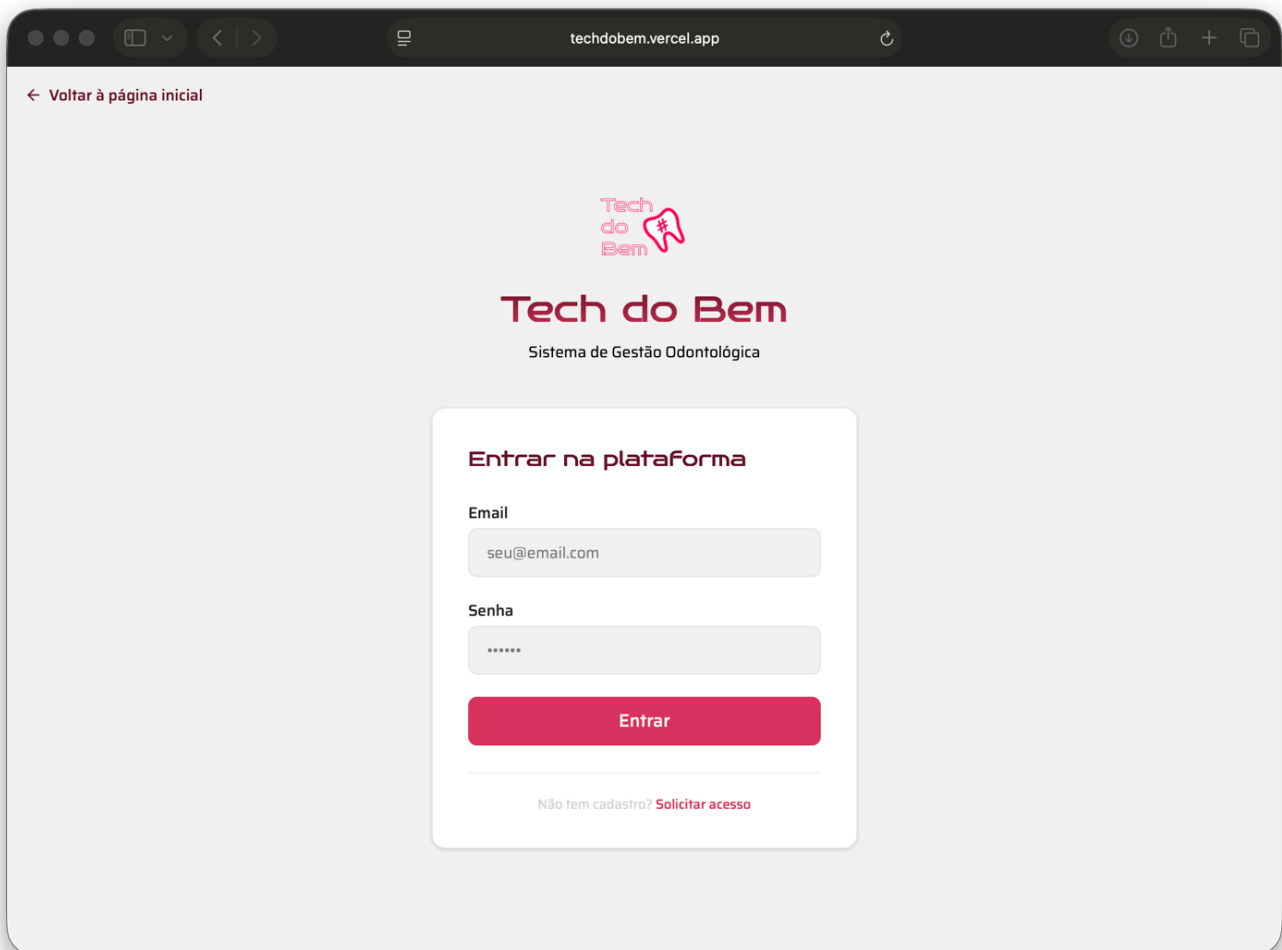
- `PreparedStatement` em 100% das queries → previne SQL Injection.
- `try-with-resources` ou `finally { close(); }` em todas as conexões.
- `setNull(idx, Types.X)` explícito para colunas anuláveis (evita o problema de `setObject(null)` no Oracle).
- `rs.getBigDecimal()` / `rs.isNull()` para coords (Oracle `getDouble` retorna `0.0` quando NULL).

5. Camada BO — regras de negócio

12 BOs encapsulam a lógica de domínio. A rubrica exige no mínimo 4 métodos com lógica de negócio relevante — entregamos muito mais. Os de maior complexidade:

Método 1 — `AuthBO.autenticar(LoginRequest)`

Busca o usuário em **duas tabelas** (`T_COLABORADOR` e `T_DENTISTA`) e devolve um `LoginResponse` polimórfico com o `role` correto. Falha com `DadoInvalidoException` se o par email/senha não bater. Permite que o frontend tenha um único formulário de login servindo dois tipos de usuário.

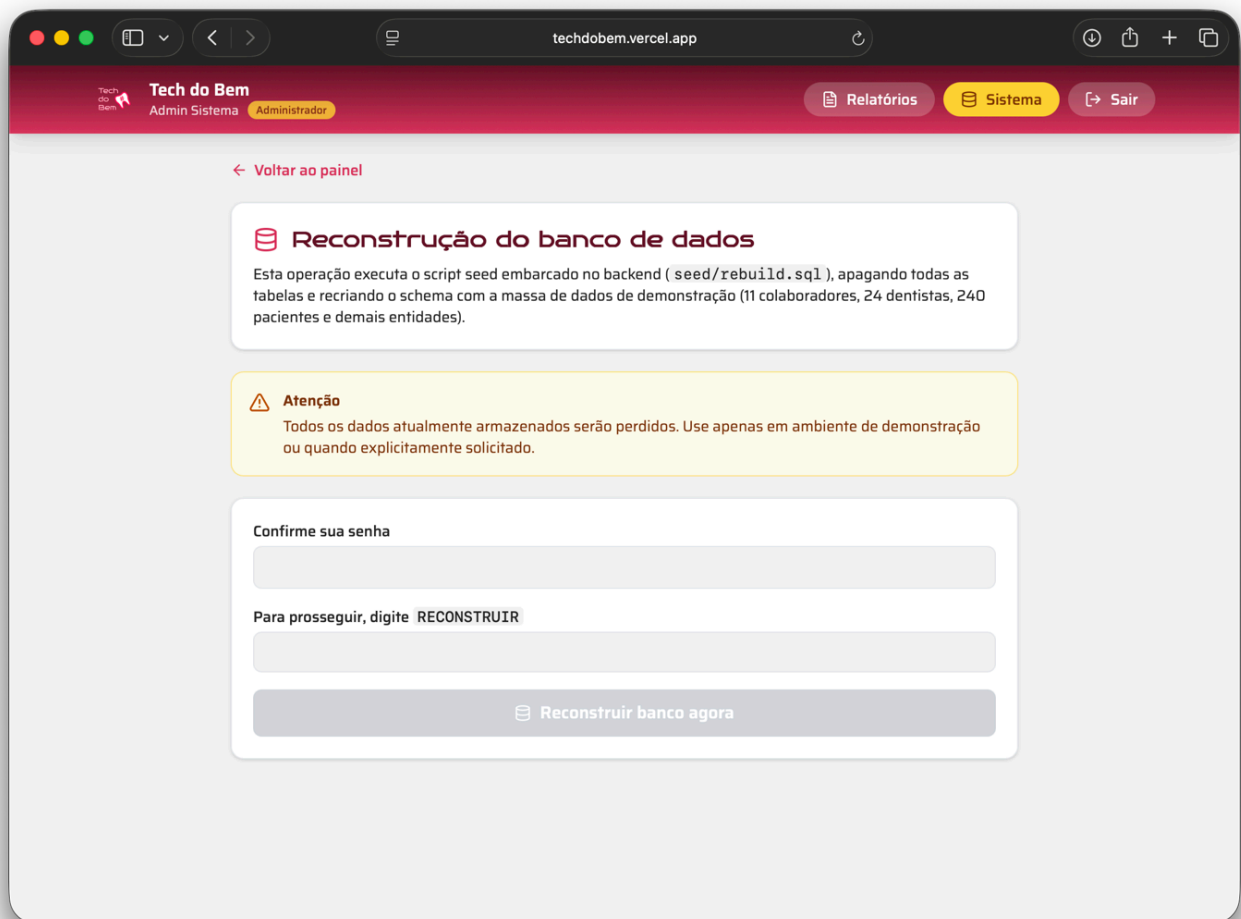


Método 2 — `AdminB0.executarRebuild(AdminRebuildRequest)`

Carrega o script SQL de `seed/rebuild.sql` do classpath e o executa via JDBC.
Implementa um **parser PL/SQL-aware** que:

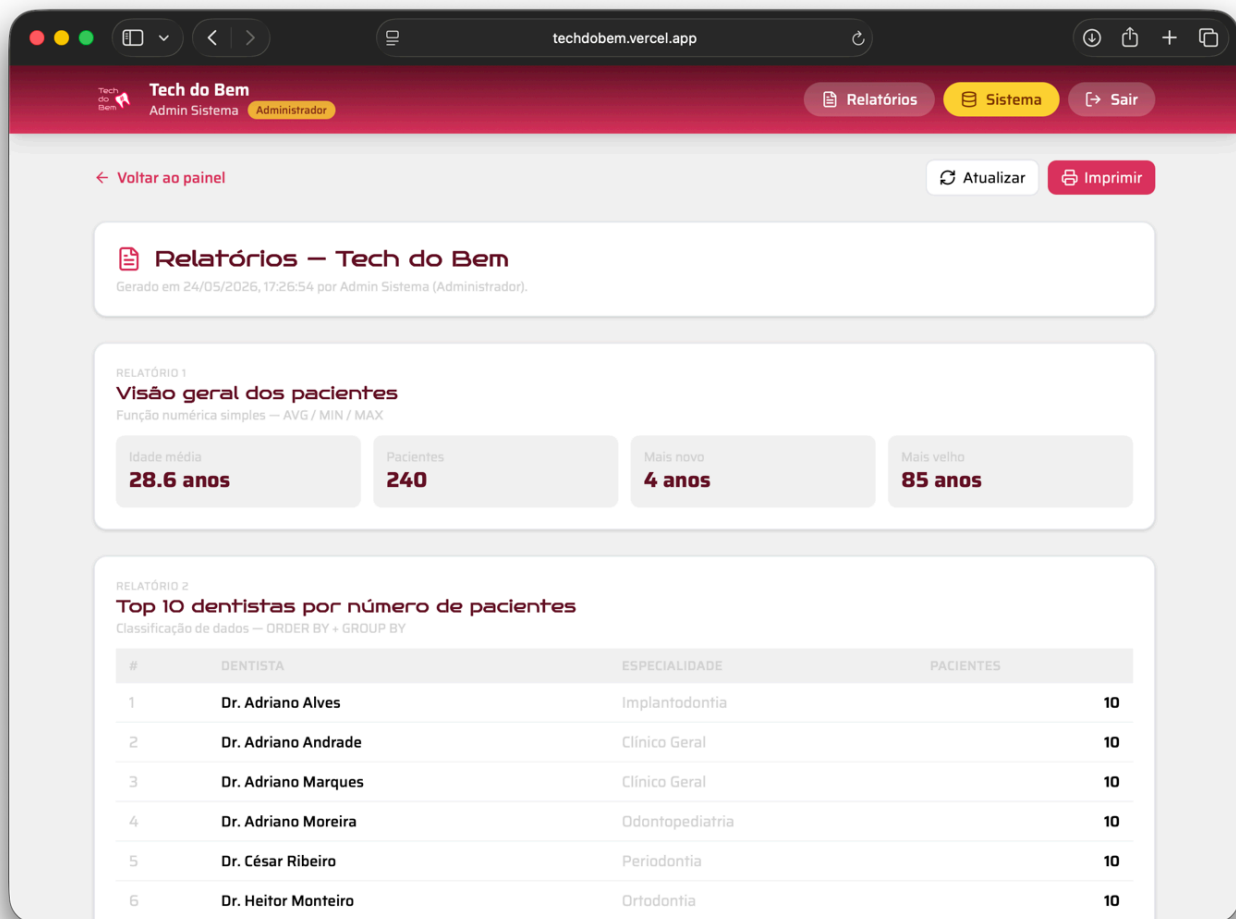
1. Detecta blocos `BEGIN...END;/` (DROPs idempotentes) e os trata como **um único statement**.
2. Pula `SELECT` s finais (os 5 relatórios da rubrica BRD são puramente demonstrativos).
3. Tolerar `ORA-00942` (tabela inexistente) e `ORA-02289` (sequence inexistente) na primeira execução.

Restrito por validação tripla: `email = admin@admin.com`, `senha = admin`, `confirmação = RECONSTRUIR`. Permite recriar o banco de demonstração a qualquer momento, atendendo ao requisito acadêmico de poder regenerar o ambiente para apresentações.



Método 3 — RelatorioB0.gerar()

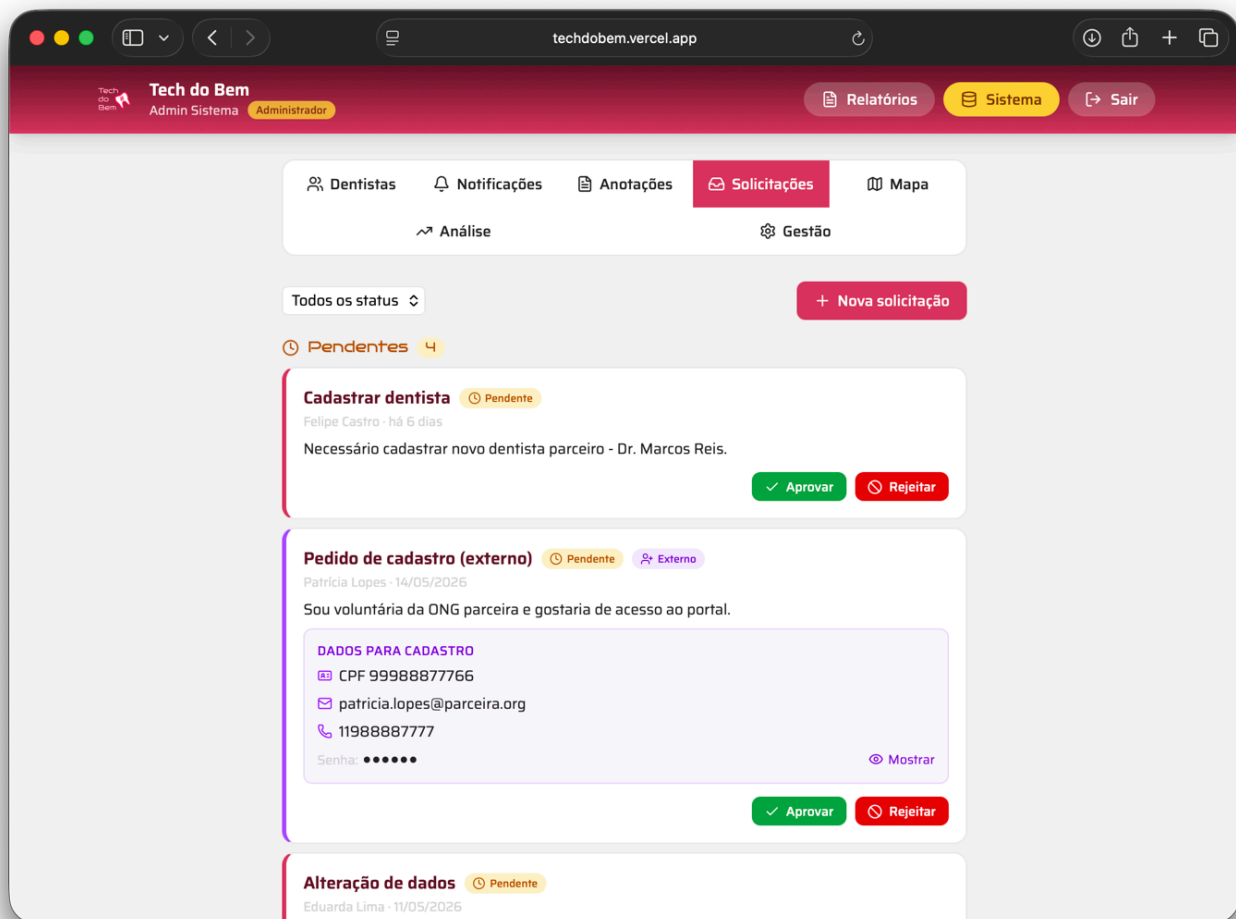
Roda os **5 SELECTs da rubrica de Banco de Dados** (classificação ORDER BY, função numérica AVG, GROUP BY com HAVING, sub-consulta, INNER JOIN triplo) em um único snapshot consolidado, retornando uma estrutura JSON com 5 chaves prontas para serem renderizadas na tela `/admin/relatorios` do frontend.



Método 4 — SolicitacaoB0.revisar(int id, SolicitacaoReviewRequest)

Aprovação ou rejeição de uma solicitação de acesso. **Lógica complexa:**

- Se for solicitação **interna** (tem `id_solicitante`): apenas marca o status (`aprovada` ou `rejeitada`).
- Se for **externa** (vinda da tela de login, sem `id_solicitante`) e for **aprovada**: cria **automaticamente um colaborador** com os dados de `nome_externo`, `cpf_externo`, `email_externo`, `senha_externo`, `telefone_externo`, e gravando o novo `id_colaborador_criado` para auditoria.
- Trata `ORA-00001` (UK violation no email) traduzindo para HTTP 409 com mensagem amigável via `SqlConflictDetector`.



Outros BOs com lógica relevante

- `PacienteBO.listarComGeo()` — filtra somente pacientes com latitude/longitude preenchidos, usado pelo mapa.
- `NotificacaoBO.marcarLida(int id)` — atualiza `data_leitura` para `SYSDATE` (PATCH idempotente).
- `AtendimentoBO` — agrega exames vinculados ao listar; valida que existe pelo menos uma campanha cadastrada antes de criar atendimento (`SEM_CAMPANHA` se vazio).
- `ColaboradorBO.atualizar` / `DentistaBO.atualizar` — preservam a senha atual quando o `request.senha` vem nulo/vazio (UPDATE parcial).

6. Camada Resource — API REST

12 resources JAX-RS expõem todos os endpoints necessários ao frontend, seguindo princípios REST:

- **URIs orientadas a recursos** (`/api/pacientes/{id}`).
- **Verbos HTTP semânticos** (GET = listar/buscar, POST = criar, PUT = atualizar inteiro, PATCH = atualizar parcial, DELETE = remover).
- **Status codes apropriados** (200, 201, 204, 400, 404, 409, 500).
- **Header Location** retornado em todo POST de criação.
- **MediaType.APPLICATION_JSON** consistente em request/response.

Todos os resources delegam a um BO único (`private final XxxBO bo = new XxxBO();`), mantendo as classes finas e fáceis de ler.

7. Tabela completa de endpoints

7.1 Autenticação

Verbo	URI	Status sucesso	Descrição
POST	<code>/api/login</code>	200	Autentica colaborador ou dentista; retorna role + id + nome + email + cargo

7.2 Administração

Verbo	URI	Status sucesso	Descrição
POST	<code>/api/admin/rebuild</code>	200	Reconstrói o banco a partir de <code>seed/rebuild.sql</code> (admin especial)
GET	<code>/api/relatorios</code>	200	Snapshot dos 5 relatórios da rubrica BRD

7.3 Colaboradores

Verbo	URI	Status sucesso
GET	/api/colaboradores	200
POST	/api/colaboradores	201
PUT	/api/colaboradores/{id}	200
DELETE	/api/colaboradores/{id}	204

7.4 Dentistas

Verbo	URI	Status sucesso
GET	/api/dentistas	200
GET	/api/dentistas/{id}	200
POST	/api/dentistas	201
PUT	/api/dentistas/{id}	200
DELETE	/api/dentistas/{id}	204

7.5 Pacientes

Verbo	URI	Status sucesso	Observação
GET	/api/pacientes	200	
GET	/api/pacientes/{id}	200	
GET	/api/pacientes/geo	200	Apenas pacientes com lat/lng
POST	/api/pacientes	201	
PUT	/api/pacientes/{id}	200	
DELETE	/api/pacientes/{id}	204	

7.6 Campanhas

Verbo	URI	Status sucesso
GET	/api/campanhas	200
POST	/api/campanhas	201
PUT	/api/campanhas/{id}	200
DELETE	/api/campanhas/{id}	204

7.7 Atendimentos

Verbo	URI	Status sucesso
GET	/api/atendimentos	200
POST	/api/atendimentos	201
PUT	/api/atendimentos/{id}	200

7.8 Exames

Verbo	URI	Status sucesso
GET	/api/exames	200
GET	/api/exames/{id}	200
POST	/api/exames	201
PUT	/api/exames/{id}	200
DELETE	/api/exames/{id}	204

7.9 Notificações

Verbo	URI	Status sucesso	Observação
GET	/api/notificacoes	200	
GET	/api/notificacoes/{id}	200	
POST	/api/notificacoes	201	

PUT	/api/notificacoes/{id}	200	
PATCH	/api/notificacoes/{id}/lida	200	Marca data_leitura = SYSDATE
DELETE	/api/notificacoes/{id}	204	

7.10 Solicitações

Verbo	URI	Status sucesso	Observação
GET	/api/solicitacoes	200	
GET	/api/solicitacoes/{id}	200	
POST	/api/solicitacoes	201	Aceita interno ou externo (tela de login)
PATCH	/api/solicitacoes/{id}/revisao	200	Aprovação cria colaborador se externo
DELETE	/api/solicitacoes/{id}	204	

7.11 Anotações

Verbo	URI	Status sucesso
GET	/api/anotacoes?sobreTipo=X&sobreId=Y	200
POST	/api/anotacoes	201
DELETE	/api/anotacoes/{id}	204

7.12 Códigos de erro comuns

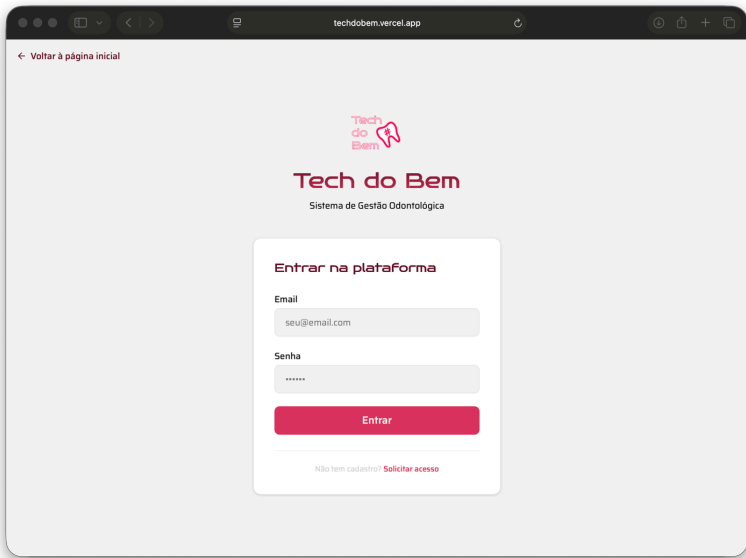
Status	Quando
	Body JSON ausente ou malformado, ou DTO com campos inválidos

400	(<code>DadoInvalidoException</code>)
404	Recurso não encontrado por ID
409	Conflito de unicidade (UK violation — ex.: email duplicado em colaborador)
500	Falha de persistência (<code>PersistenciaException</code> — envolve <code>SQLException</code> original)

8. Protótipo — telas do sistema

As capturas abaixo mostram as telas implementadas no frontend React+Vite que consomem esta API REST. Estão organizadas por personagem (Dentista / Colaborador / Admin) e ilustram o uso real dos endpoints.

8.1 Acesso

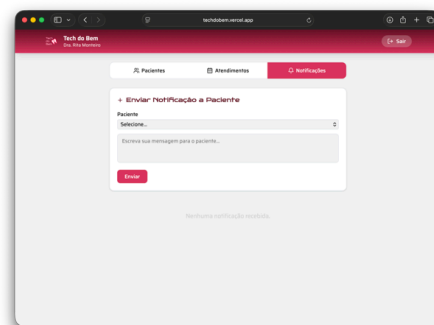
Tela	Endpoint consumido	Print
Login unificado (colaborador ou dentista)	POST <code>/api/login</code>	

8.2 Painel do Dentista

Tela	Endpoints consumidos	Print

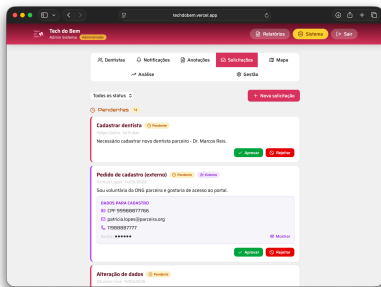
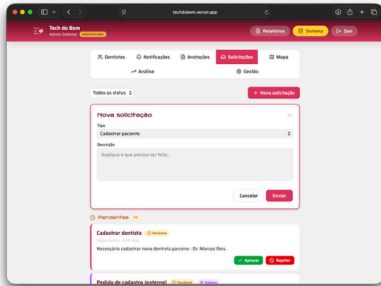
Lista de atendimentos	GET /api/atendimentos , GET /api/exames	
Modal de Registrar Atendimento	POST /api/atendimentos	
Registro com exame anexado	POST /api/atendimentos + POST /api/exames	
Pacientes vinculados	GET /api/pacientes (filtrado por dentistaId)	
Notificações recebidas / enviadas a	GET /api/notificacoes , PATCH /api/notificacoes/{id}/lida	

pacientes

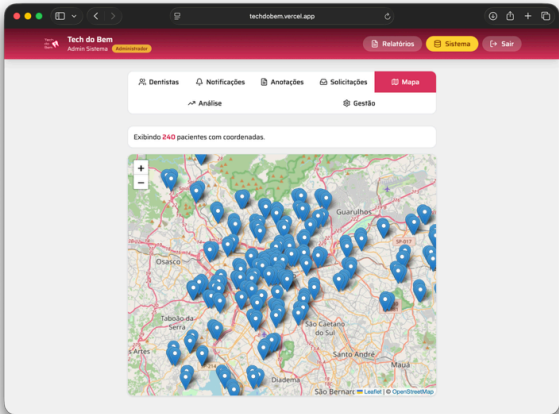


8.3 Painel do Colaborador — dia a dia

Tela	Endpoints consumidos	Print
Lista de dentistas com badge de exames pendentes	GET /api/dentistas , GET /api/exames , GET /api/atendimentos	
Notificações enviadas e status de leitura	GET /api/notificacoes	
Anotações sobre dentistas / pacientes / atendimentos	GET /api/anotacoes , POST /api/anotacoes	
Solicitações pendentes de revisão	GET /api/solicitacoes , PATCH /api/solicitacoes/{id}/revisao	

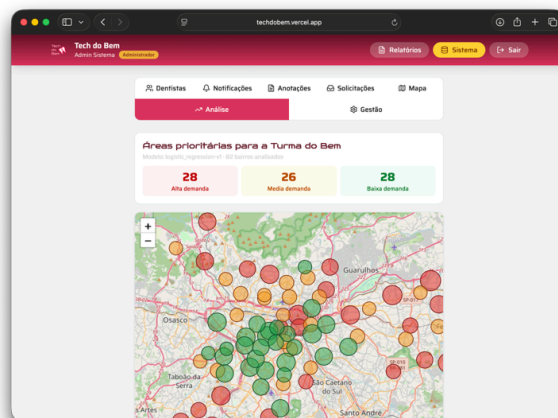
		
Nova solicitação interna	POST /api/solicitacoes	

8.4 Painel do Colaborador — análise regional (consome também o Backend_AI)

Tela	Endpoints consumidos	Print
Mapa com pacientes geolocalizados	GET /api/pacientes/geo	

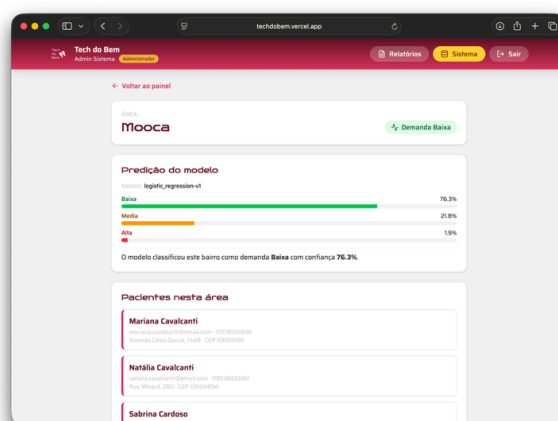
Heatmap de demanda por bairro (modelo ML)

GET
/api/pacientes/geo
+ GET /ranking
(Backend_AI)



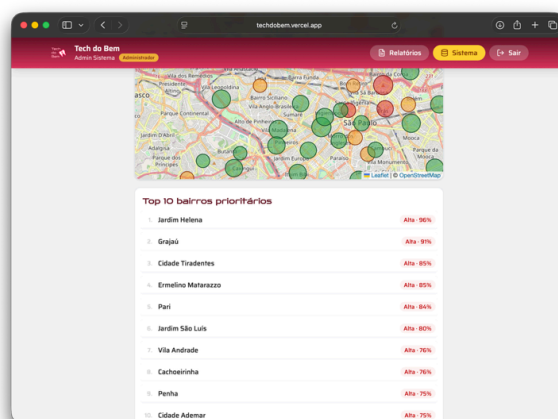
Detalhe de bairro com predição

GET
/api/pacientes ,
POST /predict
(Backend_AI)

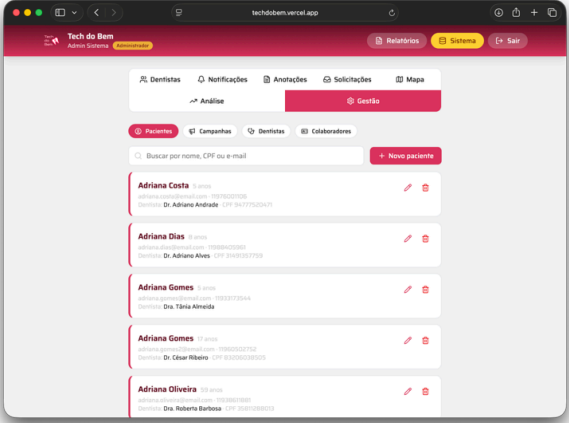
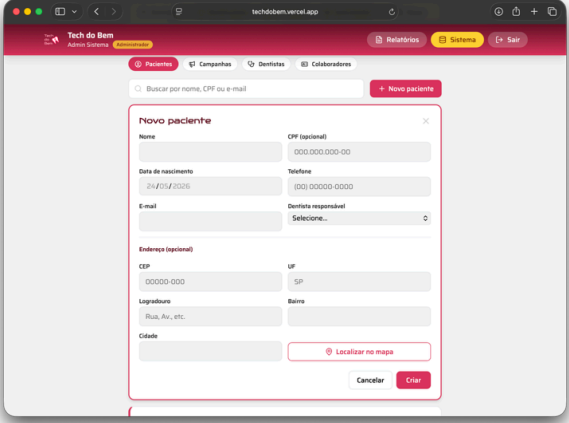
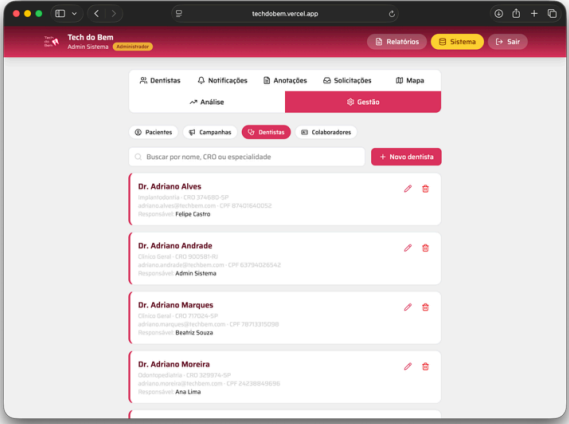


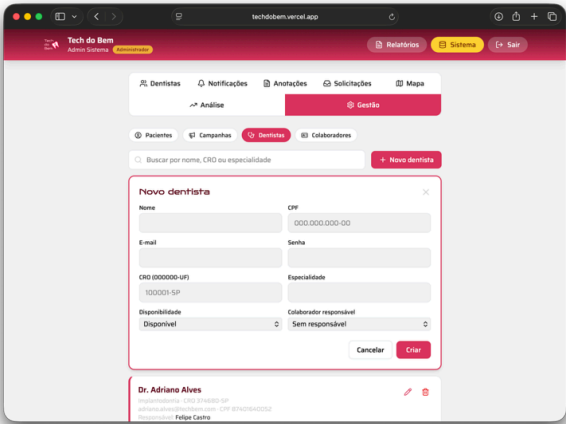
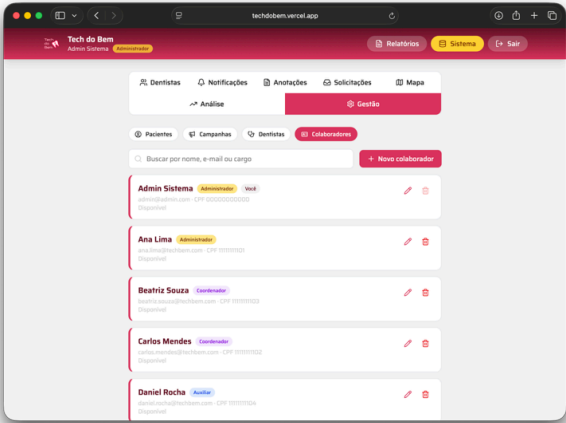
Bairros prioritários ordenados pela predição

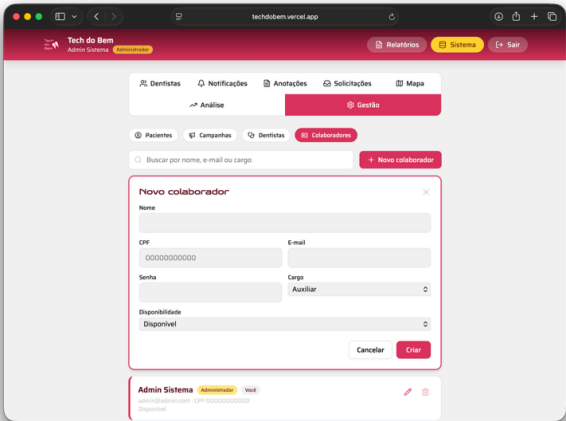
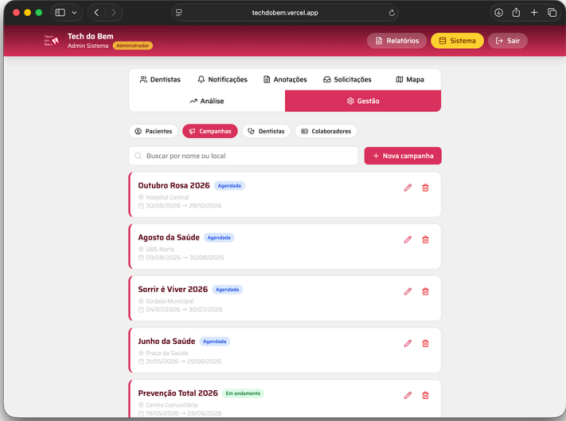
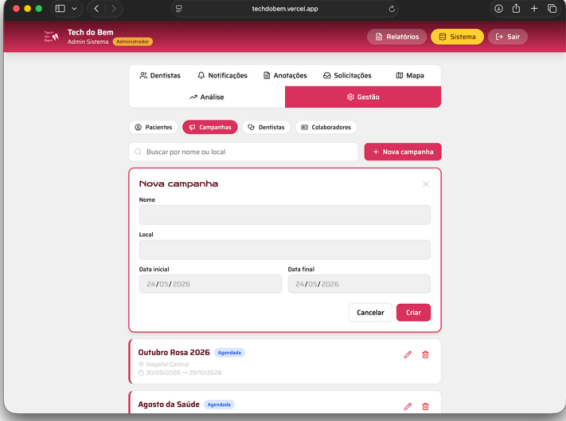
GET /ranking
(Backend_AI)



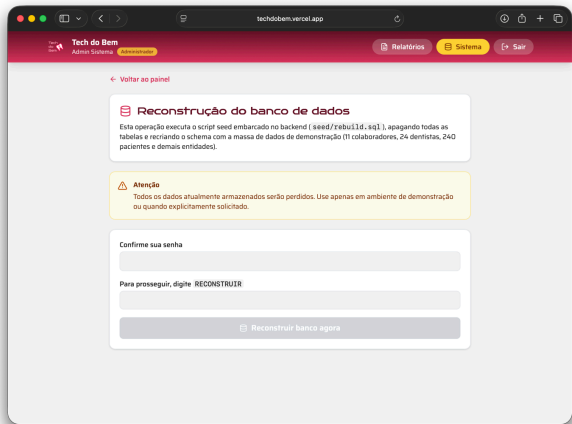
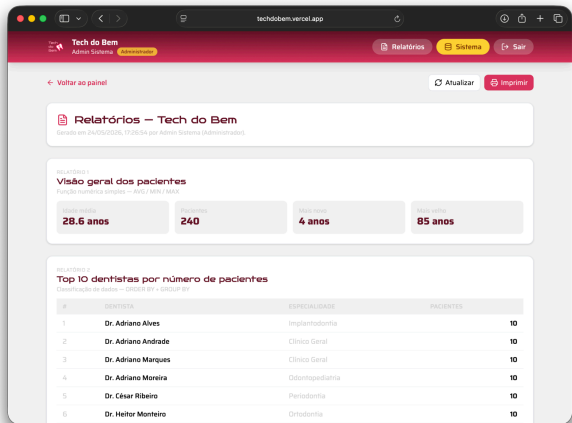
8.5 Gestão (CRUD operacional do colaborador)

Tela	Endpoints consumidos	Print
Lista de pacientes	GET /api/pacientes	
Cadastrar paciente (com ViaCEP + Nominatim)	POST /api/pacientes	
Lista de dentistas	GET /api/dentistas	

Cadastrar dentista	POST /api/dentistas	
Lista de colaboradores	GET /api/colaboradores	
Cadastrar colaborador	POST /api/colaboradores	

		
Lista de campanhas	GET /api/campanhas	
Cadastrar campanha	POST /api/campanhas	

8.6 Administração (restrito ao admin@admin.com)

Tela	Endpoints consumidos	Print
Reconstrução do banco com confirmação RECONSTRUIR	POST <code>/api/admin/rebuild</code>	
Relatórios consolidados (5 SELECTs da rubrica BRD)	GET <code>/api/relatorios</code>	

9. Tratamento de exceções

Duas exceções de domínio em `br.com.fiap.exceptions` :

9.1 DadoInvalidoException

`RuntimeException` lançada pelos BOs quando o request é semanticamente inválido (campo obrigatório vazio, valor fora do enum esperado, FK referenciando ID inexistente). Mapeada para HTTP **400 Bad Request** pelo Quarkus.


```
if (req.email() == null || req.email().isBlank()) {  
    throw new DadoInvalidoException("Email e senha sao obrigatorios."  
}
```



9.2 PersistenciaException

Envolve SQLException e ClassNotFoundException com mensagem amigável, evitando vaziar stack trace JDBC para o cliente. Mapeada para HTTP **500 Internal Server Error**.

```
catch (SQLException | ClassNotFoundException e) {  
    throw new PersistenciaException("Erro ao salvar colaborador", e)  
}
```



9.3 SqlConflictDetector

Classe utilitária que inspeciona SQLException.getErrorCode() e mapeia códigos Oracle para HTTP semântico:

- ORA-00001 (UK violation) → **409 Conflict** com mensagem "email já cadastrado" (em vez de 500 genérico).
- Reaproveitada por ColaboradorBO, DentistaBO e SolicitacaoBO durante criação/aprovação.

10. Conexão com o banco de dados

10.1 ConexaoFactory

```
public class ConexaoFactory {  
    public Connection conexao() throws ClassNotFoundException, SQLException  
        Class.forName("oracle.jdbc.driver.OracleDriver");  
    return DriverManager.getConnection(  
        "jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl",  
        System.getenv("DB_USER"),  
        System.getenv("DB_PASSWORD"));  
}
```



10.2 Credenciais

Usuário e senha são lidos de **variáveis de ambiente** (`DB_USER` , `DB_PASSWORD`) — boa prática que evita commitar credenciais. Em desenvolvimento são passadas via `export` no shell ou pela cláusula `-e` do `docker run` .

10.3 Por que JDBC puro (sem JPA / Hibernate)?

A disciplina **Building Relational Database** exige domínio explícito de SQL Oracle (constraints, sub-consultas, JOINS). Usar JPA mascararia essas estruturas. Com JDBC puro:

- Cada DAO mostra a query exata executada — coerente com o que está documentado no script SQL canônico.
- O endpoint `/api/relatorios` reaproveita textualmente os 5 SELECTs do `seed/rebuild.sql` .
- O endpoint `/api/admin/rebuild` consegue executar **DDL e PL/SQL** (DROP, CREATE, blocos `BEGIN...END;/`) que ORMs não suportam nativamente.

11. Procedimentos para rodar a aplicação

11.1 Pré-requisitos

Item	Versão
JDK	21 (Eclipse Temurin)
Maven	3.9+

Oracle	Acesso a <code>oracle.fiap.com.br:1521/orcl</code>
Variáveis de ambiente	<code>DB_USER (RM)</code> e <code>DB_PASSWORD</code>

11.2 Clone + dependências

```
git clone https://github.com/Tech-do-Bem-FIAP/Backend_Java.git
cd Backend_Java
mvn dependency:resolve
```



11.3 Modo desenvolvimento (Quarkus Dev)

```
export DB_USER=RM568542
export DB_PASSWORD=080206
```



```
mvn quarkus:dev
```

A API sobe em `http://localhost:8080` . Recarga automática a cada `mvn package` ou alteração de arquivo `.java` . Dev UI: `http://localhost:8080/q/dev` .

11.4 Build de produção

```
mvn package -DskipTests
java -jar target/quarkus-app/quarkus-run.jar
```



11.5 Docker (multi-stage)

```
docker build -t backend-java .
docker run -p 8080:8080 \
  -e DB_USER=RM568542 \
  -e DB_PASSWORD=080206 \
  backend-java
```



A imagem usa duas fases:

1. **Build stage** com `maven:3.9-eclipse-temurin-21` — baixa dependências, compila, empacota.

2. **Runtime stage** com `eclipse-temurin:21-jre` — copia somente os artefatos finais (`/lib` , `/app` , `/quarkus` , JAR principal).

Imagem final fica enxuta e portátil para qualquer servidor com Docker.

11.6 Smoke test

```
# health (qualquer recurso GET serve)
curl http://localhost:8080/api/dentistas

# autenticação como admin especial
curl -X POST http://localhost:8080/api/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@admin.com","senha":"admin"}'

# relatórios consolidados
curl http://localhost:8080/api/relatorios | jq .
```



11.7 Ferramentas recomendadas

- **IntelliJ IDEA / VS Code** com plugin Quarkus para autocompletar `application.properties` e gerar configuração de execução automática.
- **DBeaver** ou **SQL Developer** para inspecionar o Oracle durante o desenvolvimento.

12. Atendimento à rubrica

12.1 Pontuação esperada

Critério Sprint 4 (Domain Driven Design Using Java)	Pontos	Status	Evidência
Documentação (capa, sumário, escopo, funcionalidades, endpoints, MER, classes, execução)	15		Este DOCUMENTACAO.md cobre os 8 itens obrigatórios

Camada Modelo (mínimo 6 classes; entregamos 9)	10	✓	entities/ § §3
Conexão BD + métodos com lógica de negócio (mínimo 4; entregamos 4 principais + várias auxiliares)	30	✓	conexoes/ConexaoFactory.java + bo/ §5
Camadas Exceções, DAO e BO — CRUD completo	10	✓	exceptions/ , dao/ (13 DAOs), bo/ (12 BOs) §4 §5 §8
API Restful (todos endpoints necessários, princípios REST)	35	✓	resource/ (12 resources, ~45 endpoints) §6 §7
Total	100		Pontuação máxima esperada

12.2 Mitigação de penalidades

Penalidade	Mitigação
Documentação desorganizada (−5 a −20)	Sumário numerado, seções claras, tabelas para listas longas
Documentação fora de PDF (−10 a −20)	Markdown padronizado, conversível em PDF via Print do navegador
Erro de nomenclatura (−10 a −30)	Pacotes em inglês (<code>br.com.fiap.entities</code>), classes em PascalCase, métodos em camelCase, tabelas Oracle em <code>T_*</code>
Falta de organização no código (−5 a −20)	Quatro camadas estritas (entity → dao → bo → resource), sem <code>if-else</code> aninhado, métodos curtos
Baixa resolução das imagens (−5 por item)	Diagramas serão renderizados em alta resolução no PDF final

Sem link do GitHub (-35 a -85)	Link no início do README e nas instruções de execução
Código não compila (-20 a -50)	<code>mvn package</code> testado a cada commit; Dockerfile testado em CI
CRUD incompleto (-10 por item)	CRUD completo (<code>inserir</code> , <code>selecionar</code> , <code>atualizar</code> , <code>deletar</code>) em todos os DAOs principais
Falta de método de lógica (-5 a -10)	4 BOs com lógica não-trivial documentados em §5 + várias auxiliares

12.3 Decisões de design defensáveis em banca

1. **Quarkus em vez de Spring Boot:** menos boilerplate, dev UI integrada, container 3x mais leve, e tempo de boot < 1s — apropriado para deploy em servidor caseiro com Cloudflare Tunnel.
2. **JDBC puro em vez de JPA:** alinhamento direto com a disciplina BRD; o SQL exato pode ser auditado em cada DAO.
3. **DTOs como `record`** : Java 21 — imutáveis, `equals/hashCode` automáticos, serialização Jackson nativa.
4. **Sem framework de autenticação:** mantemos auth de sessão simples no frontend (`sessionStorage` do JSON do login), apropriado para o escopo acadêmico.
5. **Endpoint `/api/admin/rebuild`** : permite ao avaliador recriar o banco em qualquer momento sem precisar abrir SQL Developer — facilita correção em banca.

12.4 Repositórios irmãos do projeto Tech do Bem

- [Tech-do-Bem-FIAP/Backend_Java](#) — este repositório
- [Tech-do-Bem-FIAP/Frontend](#) — SPA React + Vite
- [Tech-do-Bem-FIAP/Backend_AI](#) — API Flask + modelo preditivo
- [Tech-do-Bem-FIAP/Database](#) — schema e seed Oracle